

CSC316 Cryptography Lab Solutions

Questions 8 to 18

Contents

1	Question 8: DES S-Box Substitution	2
2	Question 9: DES Key Generation for Initial Rounds	4
3	Question 10: Modular Arithmetic and Additive Inverse in Z_n	7
4	Question 11: Extended Euclidean Algorithm and Multiplicative Inverse	9
5	Question 12: Primality Testing using Fermat and Miller-Rabin	11
6	Question 13: Euler Totient Function and Euler's Theorem	14
7	Question 14: Primitive Root Checking and Finding Primitive Roots	16
8	Question 15: Diffie-Hellman Key Exchange	19
9	Question 16: RSA Algorithm with Key Generation, Encryption, and Decryption	21
10	Question 17: ElGamal Cryptosystem	25
11	Question 18: Safe Scanner and Replication Simulation	28

1. Question 8: DES S-Box Substitution

This program demonstrates the working of a DES S-Box. It accepts a 6-bit input, finds the row from the first and last bits, finds the column from the middle four bits, and produces the corresponding 4-bit output.

C++ Program

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main() {
6     int S1[4][16] = {
7         {14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7},
8         {0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8},
9         {4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0},
10        {15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13}
11    };
12
13    string bits;
14    cout << "Enter 6-bit input: ";
15    cin >> bits;
16
17    if (bits.length() != 6) {
18        cout << "Invalid input\n";
19        return 0;
20    }
21
22    int row = (bits[0] - '0') * 2 + (bits[5] - '0');
23    int col = (bits[1] - '0') * 8 + (bits[2] - '0') * 4 + (bits
24        [3] - '0') * 2 + (bits[4] - '0');
25    int value = S1[row][col];
26
27    string output = "";
28    for (int i = 3; i >= 0; i--) {
29        output += char(((value >> i) & 1) + '0');
30    }
31
32    cout << "Row = " << row << "\n";
33    cout << "Column = " << col << "\n";
```

```
33     cout << "Decimal Output = " << value << "\n";
34     cout << "4-bit Output = " << output << "\n";
35
36     return 0;
37 }
```

Sample Run

Input: 011011

Row = $01_2 = 1$, Column = $1101_2 = 13$, so the output is 0101.

2. Question 9: DES Key Generation for Initial Rounds

This program implements DES key generation. It applies PC-1, splits the key into two halves, performs left shifts based on the DES shift schedule, and then applies PC-2 to generate round subkeys.

C++ Program

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int PC1[56] = {
6     57,49,41,33,25,17,9,
7     1,58,50,42,34,26,18,
8     10,2,59,51,43,35,27,
9     19,11,3,60,52,44,36,
10    63,55,47,39,31,23,15,
11    7,62,54,46,38,30,22,
12    14,6,61,53,45,37,29,
13    21,13,5,28,20,12,4
14 };
15
16 int PC2[48] = {
17     14,17,11,24,1,5,
18     3,28,15,6,21,10,
19     23,19,12,4,26,8,
20     16,7,27,20,13,2,
21     41,52,31,37,47,55,
22     30,40,51,45,33,48,
23     44,49,39,56,34,53,
24     46,42,50,36,29,32
25 };
26
27 int shifts[16] = {1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2,
28     1};
29
30 string permute(string input, int table[], int n) {
31     string output = "";
32     for (int i = 0; i < n; i++) {
33         output += input[table[i] - 1];
34     }
35     return output;
36 }
```

```
33     }
34     return output;
35 }
36
37 string leftShift(string s, int count) {
38     while (count--) {
39         s = s.substr(1) + s[0];
40     }
41     return s;
42 }
43
44 int main() {
45     string key64;
46     int rounds;
47
48     cout << "Enter 64-bit key: ";
49     cin >> key64;
50     cout << "Enter number of rounds to generate: ";
51     cin >> rounds;
52
53     if (key64.length() != 64 || rounds < 1 || rounds > 16) {
54         cout << "Invalid input\n";
55         return 0;
56     }
57
58     string key56 = permute(key64, PC1, 56);
59     string C = key56.substr(0, 28);
60     string D = key56.substr(28, 28);
61
62     for (int i = 0; i < rounds; i++) {
63         C = leftShift(C, shifts[i]);
64         D = leftShift(D, shifts[i]);
65         string combined = C + D;
66         string subkey = permute(combined, PC2, 48);
67
68         cout << "Round " << i + 1 << " Subkey: " << subkey << "\n";
69     }
70
71     return 0;
72 }
```

Logic

The 64-bit key is first reduced to 56 bits using PC-1. Then the 56-bit key is divided into two 28-bit halves. For each round, both halves are shifted left according to the DES schedule and compressed to 48 bits using PC-2.

3. Question 10: Modular Arithmetic and Additive Inverse in Z_n

This program performs modular addition, subtraction, multiplication, and computes additive inverses for the given values in Z_n .

C++ Program

```
1 #include <iostream>
2 using namespace std;
3
4 long long normalize(long long x, long long n) {
5     x %= n;
6     if (x < 0) x += n;
7     return x;
8 }
9
10 int main() {
11     long long a, b, n;
12     cout << "Enter a, b and n: ";
13     cin >> a >> b >> n;
14
15     if (n <= 0) {
16         cout << "n must be positive\n";
17         return 0;
18     }
19
20     long long A = normalize(a, n);
21     long long B = normalize(b, n);
22
23     cout << "(a + b) mod n = " << normalize(A + B, n) << "\n";
24     cout << "(a - b) mod n = " << normalize(A - B, n) << "\n";
25     cout << "(a * b) mod n = " << normalize(A * B, n) << "\n";
26     cout << "Additive inverse of a in Z_" << n << " = " <<
        normalize(-A, n) << "\n";
27     cout << "Additive inverse of b in Z_" << n << " = " <<
        normalize(-B, n) << "\n";
28
29     return 0;
30 }
```

Example

If $a = 17$, $b = 23$, and $n = 10$, then:

$$17 \equiv 7 \pmod{10}, \quad 23 \equiv 3 \pmod{10}$$

$$(a + b) \bmod 10 = 0, \quad (a - b) \bmod 10 = 4, \quad (ab) \bmod 10 = 1$$

The additive inverse of 7 in Z_{10} is 3 because $7 + 3 \equiv 0 \pmod{10}$.

4. Question 11: Extended Euclidean Algorithm and Multiplicative Inverse

This program computes $\gcd(a, n)$ using the Extended Euclidean Algorithm and finds the multiplicative inverse of a modulo n when it exists.

C++ Program

```
1 #include <iostream>
2 using namespace std;
3
4 long long extendedGCD(long long a, long long b, long long &x,
5     long long &y) {
6     if (b == 0) {
7         x = 1;
8         y = 0;
9         return a;
10    }
11
12    long long x1, y1;
13    long long gcd = extendedGCD(b, a % b, x1, y1);
14
15    x = y1;
16    y = x1 - (a / b) * y1;
17
18    return gcd;
19 }
20
21 int main() {
22     long long a, n, x, y;
23     cout << "Enter a and n: ";
24     cin >> a >> n;
25
26     long long gcd = extendedGCD(a, n, x, y);
27
28     cout << "gcd(" << a << ", " << n << ") = " << gcd << "\n";
29
30     if (gcd == 1) {
31         long long inverse = (x % n + n) % n;
32         cout << "Multiplicative inverse of " << a << " mod " << n
33             << " = " << inverse << "\n";
```

```
32     } else {  
33         cout << "Multiplicative inverse does not exist\n";  
34     }  
35  
36     return 0;  
37 }
```

Logic

If $\gcd(a, n) = 1$, then the inverse exists. The Extended Euclidean Algorithm gives integers x and y such that:

$$ax + ny = \gcd(a, n)$$

So when the gcd is 1, $ax + ny = 1$, and therefore x is the multiplicative inverse of a modulo n .

5. Question 12: Primality Testing using Fermat and Miller-Rabin

This program checks whether a number is prime by using both the Fermat primality test and the Miller-Rabin test. For student laboratory use, it works well for ordinary sample inputs.

C++ Program

```
1 #include <iostream>
2 using namespace std;
3
4 long long powerMod(long long a, long long b, long long mod) {
5     long long result = 1;
6     a %= mod;
7
8     while (b > 0) {
9         if (b % 2 == 1) result = (result * a) % mod;
10        a = (a * a) % mod;
11        b /= 2;
12    }
13
14    return result;
15 }
16
17 bool fermatTest(long long n) {
18     if (n < 4) return n == 2 || n == 3;
19
20     int bases[] = {2, 3, 5, 7};
21     for (int i = 0; i < 4; i++) {
22         if (bases[i] >= n) continue;
23         if (powerMod(bases[i], n - 1, n) != 1) return false;
24     }
25
26     return true;
27 }
28
29 bool millerRabin(long long n) {
30     if (n < 2) return false;
31     if (n == 2 || n == 3) return true;
32     if (n % 2 == 0) return false;
```

```
34     long long d = n - 1;
35     int s = 0;
36
37     while (d % 2 == 0) {
38         d /= 2;
39         s++;
40     }
41
42     int bases[] = {2, 3, 5, 7, 11};
43     for (int i = 0; i < 5; i++) {
44         long long a = bases[i];
45         if (a >= n) continue;
46
47         long long x = powerMod(a, d, n);
48         if (x == 1 || x == n - 1) continue;
49
50         bool witness = true;
51         for (int r = 1; r < s; r++) {
52             x = (x * x) % n;
53             if (x == n - 1) {
54                 witness = false;
55                 break;
56             }
57         }
58
59         if (witness) return false;
60     }
61
62     return true;
63 }
64
65 int main() {
66     long long n;
67     cout << "Enter number: ";
68     cin >> n;
69
70     cout << "Fermat Test: ";
71     if (fermatTest(n)) cout << "Probably Prime\n";
72     else cout << "Composite\n";
73
74     cout << "Miller-Rabin Test: ";
```

```
75     if (millerRabin(n)) cout << "Probably Prime\n";
76     else cout << "Composite\n";
77
78     return 0;
79 }
```

Note

Miller-Rabin is stronger than Fermat because some composite numbers can still pass the Fermat test.

6. Question 13: Euler Totient Function and Euler's Theorem

This program computes Euler's totient function $\phi(n)$ and verifies Euler's theorem:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

when $\gcd(a, n) = 1$.

C++ Program

```
1 #include <iostream>
2 using namespace std;
3
4 long long gcdValue(long long a, long long b) {
5     while (b != 0) {
6         long long t = a % b;
7         a = b;
8         b = t;
9     }
10    return a;
11 }
12
13 long long powerMod(long long a, long long b, long long mod) {
14     long long result = 1;
15     a %= mod;
16
17     while (b > 0) {
18         if (b % 2 == 1) result = (result * a) % mod;
19         a = (a * a) % mod;
20         b /= 2;
21     }
22
23     return result;
24 }
25
26 long long phi(long long n) {
27     long long result = n;
28
29     for (long long p = 2; p * p <= n; p++) {
30         if (n % p == 0) {
31             while (n % p == 0) n /= p;
```

```
32         result -= result / p;
33     }
34 }
35
36     if (n > 1) result -= result / n;
37     return result;
38 }
39
40 int main() {
41     long long a, n;
42     cout << "Enter a and n: ";
43     cin >> a >> n;
44
45     long long phiValue = phi(n);
46     cout << "phi(" << n << ") = " << phiValue << "\n";
47
48     if (gcdValue(a, n) == 1) {
49         cout << "a^phi(n) mod n = " << powerMod(a, phiValue, n)
50             << "\n";
51         if (powerMod(a, phiValue, n) == 1) {
52             cout << "Euler's theorem verified\n";
53         } else {
54             cout << "Euler's theorem not verified\n";
55         }
56     } else {
57         cout << "gcd(a, n) != 1, so Euler's theorem is not
58             applicable\n";
59     }
60     return 0;
61 }
```

Example

For $a = 3$ and $n = 10$:

$$\phi(10) = 4, \quad 3^4 = 81, \quad 81 \bmod 10 = 1$$

So Euler's theorem is verified.

7. Question 14: Primitive Root Checking and Finding Primitive Roots

For a prime modulus p , a number g is a primitive root if its powers generate all non-zero residues modulo p . This program checks whether a given number is a primitive root and also lists all primitive roots of the given prime.

C++ Program

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 long long powerMod(long long a, long long b, long long mod) {
6     long long result = 1;
7     a %= mod;
8
9     while (b > 0) {
10         if (b % 2 == 1) result = (result * a) % mod;
11         a = (a * a) % mod;
12         b /= 2;
13     }
14
15     return result;
16 }
17
18 bool isPrime(long long n) {
19     if (n < 2) return false;
20     for (long long i = 2; i * i <= n; i++) {
21         if (n % i == 0) return false;
22     }
23     return true;
24 }
25
26 vector<long long> primeFactors(long long n) {
27     vector<long long> factors;
28
29     for (long long i = 2; i * i <= n; i++) {
30         if (n % i == 0) {
31             factors.push_back(i);
```



```
32         while (n % i == 0) n /= i;
33     }
34 }
35
36     if (n > 1) factors.push_back(n);
37     return factors;
38 }
39
40 bool isPrimitiveRoot(long long g, long long p, vector<long long>
    factors) {
41     long long phi = p - 1;
42
43     for (int i = 0; i < (int)factors.size(); i++) {
44         if (powerMod(g, phi / factors[i], p) == 1) return false;
45     }
46
47     return true;
48 }
49
50 int main() {
51     long long p, g;
52     cout << "Enter prime modulus p: ";
53     cin >> p;
54     cout << "Enter value g to check: ";
55     cin >> g;
56
57     if (!isPrime(p)) {
58         cout << "p must be prime\n";
59         return 0;
60     }
61
62     vector<long long> factors = primeFactors(p - 1);
63
64     if (isPrimitiveRoot(g, p, factors)) {
65         cout << g << " is a primitive root modulo " << p << "\n";
66     } else {
67         cout << g << " is not a primitive root modulo " << p << "
            \n";
68     }
69
70     cout << "Primitive roots modulo " << p << ": ";
```

```
71     for (long long i = 2; i < p; i++) {  
72         if (isPrimitiveRoot(i, p, factors)) {  
73             cout << i << " ";  
74         }  
75     }  
76     cout << "\n";  
77  
78     return 0;  
79 }
```

Logic

If p is prime, then the order of every non-zero element divides $p - 1$. To test whether g is a primitive root, it is enough to check:

$$g^{(p-1)/q} \not\equiv 1 \pmod{p}$$

for every prime factor q of $p - 1$.

8. Question 15: Diffie-Hellman Key Exchange

This program demonstrates the Diffie-Hellman key exchange. Two users choose private keys, generate public values, and finally compute the same shared secret.

C++ Program

```
1 #include <iostream>
2 using namespace std;
3
4 long long powerMod(long long a, long long b, long long mod) {
5     long long result = 1;
6     a %= mod;
7
8     while (b > 0) {
9         if (b % 2 == 1) result = (result * a) % mod;
10        a = (a * a) % mod;
11        b /= 2;
12    }
13
14    return result;
15 }
16
17 int main() {
18     long long p, g, a, b;
19     cout << "Enter prime p and primitive root g: ";
20     cin >> p >> g;
21     cout << "Enter private key of Alice: ";
22     cin >> a;
23     cout << "Enter private key of Bob: ";
24     cin >> b;
25
26     long long A = powerMod(g, a, p);
27     long long B = powerMod(g, b, p);
28
29     long long sharedAlice = powerMod(B, a, p);
30     long long sharedBob = powerMod(A, b, p);
31
32     cout << "Alice public value = " << A << "\n";
33     cout << "Bob public value = " << B << "\n";
```

```
34     cout << "Shared key computed by Alice = " << sharedAlice << "\n";
35     cout << "Shared key computed by Bob = " << sharedBob << "\n";
36
37     return 0;
38 }
```

Example

Let $p = 23$, $g = 5$, Alice's private key $a = 6$, and Bob's private key $b = 15$.

Then:

$$A = 5^6 \bmod 23 = 8, \quad B = 5^{15} \bmod 23 = 19$$

$$K_A = 19^6 \bmod 23 = 2, \quad K_B = 8^{15} \bmod 23 = 2$$

Both users get the same secret key.

9. Question 16: RSA Algorithm with Key Generation, Encryption, and Decryption

This program implements RSA using small primes. It generates the public and private keys, encrypts a text message using simple alphabet mapping, and then decrypts it.

Mapping Used

$$A \rightarrow 0, B \rightarrow 1, C \rightarrow 2, \dots, Z \rightarrow 25$$

C++ Program

```
1 #include <iostream>
2 #include <string>
3 #include <cctype>
4 using namespace std;
5
6 long long gcdValue(long long a, long long b) {
7     while (b != 0) {
8         long long t = a % b;
9         a = b;
10        b = t;
11    }
12    return a;
13 }
14
15 long long powerMod(long long a, long long b, long long mod) {
16     long long result = 1;
17     a %= mod;
18
19     while (b > 0) {
20         if (b % 2 == 1) result = (result * a) % mod;
21         a = (a * a) % mod;
22         b /= 2;
23     }
24
25     return result;
26 }
27
```

```
28 long long extendedGCD(long long a, long long b, long long &x,  
    long long &y) {  
29     if (b == 0) {  
30         x = 1;  
31         y = 0;  
32         return a;  
33     }  
34  
35     long long x1, y1;  
36     long long gcd = extendedGCD(b, a % b, x1, y1);  
37  
38     x = y1;  
39     y = x1 - (a / b) * y1;  
40  
41     return gcd;  
42 }  
43  
44 long long modInverse(long long a, long long mod) {  
45     long long x, y;  
46     long long gcd = extendedGCD(a, mod, x, y);  
47     if (gcd != 1) return -1;  
48     return (x % mod + mod) % mod;  
49 }  
50  
51 int main() {  
52     long long p, q;  
53     cout << "Enter two prime numbers p and q: ";  
54     cin >> p >> q;  
55  
56     long long n = p * q;  
57     long long phi = (p - 1) * (q - 1);  
58  
59     if (n <= 25) {  
60         cout << "Choose larger primes so that n > 25\n";  
61         return 0;  
62     }  
63  
64     long long e = 2;  
65     while (e < phi && gcdValue(e, phi) != 1) e++;  
66  
67     long long d = modInverse(e, phi);
```

```
68
69     cout << "Public key (e, n) = (" << e << ", " << n << ")\n";
70     cout << "Private key (d, n) = (" << d << ", " << n << ")\n";
71
72     cin.ignore();
73     string message;
74     cout << "Enter message in uppercase letters: ";
75     getline(cin, message);
76
77     cout << "Encrypted values: ";
78     for (int i = 0; i < (int)message.length(); i++) {
79         char ch = toupper(message[i]);
80
81         if (ch == ' ') {
82             cout << "/ ";
83         } else if (ch >= 'A' && ch <= 'Z') {
84             long long m = ch - 'A';
85             long long c = powerMod(m, e, n);
86             cout << c << " ";
87         }
88     }
89     cout << "\n";
90
91     cout << "Decrypted message: ";
92     for (int i = 0; i < (int)message.length(); i++) {
93         char ch = toupper(message[i]);
94
95         if (ch == ' ') {
96             cout << " ";
97         } else if (ch >= 'A' && ch <= 'Z') {
98             long long m = ch - 'A';
99             long long c = powerMod(m, e, n);
100             long long plain = powerMod(c, d, n);
101             cout << char(plain + 'A');
102         }
103     }
104     cout << "\n";
105
106     return 0;
107 }
```

Logic

First compute:

$$n = pq, \quad \phi(n) = (p-1)(q-1)$$

Choose e such that $\gcd(e, \phi(n)) = 1$, and compute d such that:

$$ed \equiv 1 \pmod{\phi(n)}$$

Encryption is:

$$C = M^e \bmod n$$

Decryption is:

$$M = C^d \bmod n$$

10. Question 17: ElGamal Cryptosystem

This program implements ElGamal key generation, encryption, and decryption for a sample integer message.

C++ Program

```
1 #include <iostream>
2 using namespace std;
3
4 long long powerMod(long long a, long long b, long long mod) {
5     long long result = 1;
6     a %= mod;
7
8     while (b > 0) {
9         if (b % 2 == 1) result = (result * a) % mod;
10        a = (a * a) % mod;
11        b /= 2;
12    }
13
14    return result;
15 }
16
17 long long extendedGCD(long long a, long long b, long long &x,
18     long long &y) {
19     if (b == 0) {
20         x = 1;
21         y = 0;
22         return a;
23     }
24
25     long long x1, y1;
26     long long gcd = extendedGCD(b, a % b, x1, y1);
27
28     x = y1;
29     y = x1 - (a / b) * y1;
30
31     return gcd;
32 }
33
34 long long modInverse(long long a, long long mod) {
```

```
34     long long x, y;
35     long long gcd = extendedGCD(a, mod, x, y);
36     if (gcd != 1) return -1;
37     return (x % mod + mod) % mod;
38 }
39
40 int main() {
41     long long p, g, x, k, m;
42     cout << "Enter prime p and primitive root g: ";
43     cin >> p >> g;
44     cout << "Enter private key x: ";
45     cin >> x;
46     cout << "Enter random key k: ";
47     cin >> k;
48     cout << "Enter message m (m < p): ";
49     cin >> m;
50
51     long long y = powerMod(g, x, p);
52
53     long long c1 = powerMod(g, k, p);
54     long long c2 = (m * powerMod(y, k, p)) % p;
55
56     long long s = powerMod(c1, x, p);
57     long long sInverse = modInverse(s, p);
58     long long decrypted = (c2 * sInverse) % p;
59
60     cout << "Public key (p, g, y) = (" << p << ", " << g << ", "
        << y << ")\n";
61     cout << "Ciphertext (c1, c2) = (" << c1 << ", " << c2 << ")\n";
62     cout << "Decrypted message = " << decrypted << "\n";
63
64     return 0;
65 }
```

Logic

If the private key is x , then the public key is:

$$y = g^x \bmod p$$

For encryption with random key k :

$$c_1 = g^k \bmod p, \quad c_2 = m \cdot y^k \bmod p$$

For decryption:

$$m = c_2 \cdot (c_1^x)^{-1} \bmod p$$

11. Question 18: Safe Scanner and Replication Simulation

The original lab question asks for a harmless demonstration of scanner or replication behavior. The following program is intentionally safe. It does not access real files, does not scan networks, does not copy itself, and does not make any system changes. It only simulates the logic in memory for classroom understanding.

C++ Program

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 using namespace std;
5
6 bool hasSuspiciousExtension(string name) {
7     string exts[] = {".exe", ".bat", ".vbs", ".js", ".scr", ".com",
8         ""};
9     for (int i = 0; i < 6; i++) {
10         if (name.size() >= exts[i].size()) {
11             if (name.substr(name.size() - exts[i].size()) == exts
12                 [i]) {
13                 return true;
14             }
15         }
16     }
17     return false;
18 }
19
20 int main() {
21     int n;
22     cout << "Enter number of sandbox file names: ";
23     cin >> n;
24
25     vector<string> files(n);
26     cout << "Enter file names:\n";
27     for (int i = 0; i < n; i++) {
28         cin >> files[i];
29     }
30
31     cout << "\nScan Report\n";
32     for (int i = 0; i < n; i++) {

```

```
31     if (hasSuspiciousExtension(files[i])) {
32         cout << files[i] << " -> suspicious pattern detected\n";
33     } else {
34         cout << files[i] << " -> normal\n";
35     }
36 }
37
38 string agentName;
39 int copies;
40 cout << "\nEnter a demo agent name: ";
41 cin >> agentName;
42 cout << "Enter number of virtual copies to simulate: ";
43 cin >> copies;
44
45 vector<string> sandboxCopies;
46 for (int i = 1; i <= copies; i++) {
47     sandboxCopies.push_back("copy_" + to_string(i) + "_" +
48                             agentName);
49 }
50
51 cout << "\nVirtual Replication Log\n";
52 for (int i = 0; i < (int)sandboxCopies.size(); i++) {
53     cout << sandboxCopies[i] << "\n";
54 }
55
56 cout << "\nThis was only an in-memory classroom simulation.\n";
57
58 return 0;
59 }
```

Why this version is safe

It demonstrates the idea of pattern matching and replication flow without touching the operating system, network, registry, or real files. So students can understand the concept without creating harmful behavior.